

Summary and Reflections Report

CS-320: Software Test Automation and QA

Sydney Brown

April 19, 2026

Unit Testing Approach and Alignment to Requirements

When I built the three service components for Project One, my overall approach to unit testing was to write tests that directly mapped to the requirements specified in the project guidelines. Each service had a defined set of operations it needed to support, and I structured my JUnit tests around those operations one by one so that nothing was left untested. The goal was not just to make the code run, but to verify that it behaved correctly under both normal and edge-case conditions.

For the `ContactService`, the requirements specified that each contact must have a unique ID no longer than ten characters, and that fields like first name, last name, phone number, and address all had defined length and null constraints. My tests were written to check each of those rules explicitly. For example, `testAddContact()` created a valid contact with ID "ABC123" and verified it could be retrieved by calling `service.getContact("ABC123").getFirstName()` and asserting it equaled "Sydney." That directly confirmed the add operation worked as required. To cover the constraint that duplicate IDs cannot be added, `testAddDuplicateContact()` attempted to add two contacts sharing the same ID and used `assertThrows(IllegalArgumentException.class, ...)` to confirm the service rejected the second entry. This pattern of testing both the happy path and the rejection path was how I kept my tests aligned with what the requirements actually demanded.

The `TaskService` requirements were similar in structure: tasks needed a unique ID, a name no longer than twenty characters, and a description no longer than fifty characters, with no null fields allowed. The `AppointmentService` introduced a date constraint, requiring that appointment dates not be in the past. To handle this cleanly and repeatably, I wrote a

futureDate() helper method in `AppointmentServiceTest` that returned **new Date(System.currentTimeMillis() + 86400000L)**, which is always one day ahead of the current time. This made sure my tests would never accidentally fail just because a hardcoded date became stale. That kind of deliberate design decision is one of the ways I kept my testing approach aligned to the requirements rather than just testing whatever was convenient.

Across all three services, I also tested deletion and update behavior to match the requirement that those operations should throw an exception when an ID does not exist. For instance, **testDeleteNonExistentAppointment()** called **service.deleteAppointment("NONE")** and verified that an **IllegalArgumentException** was thrown, which confirms the service enforces the requirement rather than silently ignoring a bad input.

Effectiveness of JUnit Tests

The effectiveness of my JUnit tests is supported by the coverage percentages produced by Eclipse's built-in coverage tool. `ContactService` achieved 81.4% instruction coverage, `TaskService` achieved 88.4%, and `AppointmentService` achieved 86.3%. All three exceed the 80% threshold required by the project, which tells me the tests are reaching the majority of the code paths in each service class.

Coverage percentages alone do not prove that tests are good, but in this case the numbers are backed up by the structure of the tests themselves. For each service, I wrote tests for every core operation: adding valid entries, rejecting duplicates, deleting entries, rejecting deletions of nonexistent entries, and in the case of `Contact` and `Task`, updating individual fields. The coverage reflects that breadth. The fact that no tests failed and all assertions passed across every run confirms that the code meets its requirements and the tests are functioning as intended.

Writing the JUnit Tests: Technical Soundness and Efficiency

To ensure my code was technically sound, I relied on JUnit 5's assertion and exception-testing tools consistently and correctly throughout all three test files. One example of this is how I handled exception testing. Rather than wrapping code in try-catch blocks, which can mask problems if an exception is thrown at the wrong moment, I used **assertThrows()** with a lambda expression, which is the recommended approach for verifying exceptions in JUnit 5 (JUnit Team, 2024). In `ContactServiceTest`, the duplicate ID test reads:

```
assertThrows(IllegalArgumentException.class, () -> service.addContact(contact2));
```

This is the correct, modern JUnit 5 approach to verifying that a specific exception is thrown by a specific operation. It is precise and cannot pass accidentally. I used this same pattern across `ContactServiceTest`, `TaskServiceTest`, and `AppointmentServiceTest` wherever a rejection scenario needed to be tested, which is what made those tests technically sound rather than just present. I also used **@BeforeEach** in every test class to reset the service instance before each test method ran. In `AppointmentServiceTest` this looks like:

```
@BeforeEach  
  
void setUp() {  
  
    service = new AppointmentService();  
  
}
```

This ensured that no test was able to influence the state seen by another test, which is a fundamental requirement for reliable unit tests. Without this, a test that added a record could cause a later test to fail because the record already existed.

For efficiency, I kept each test method focused on a single behavior. For example, **testAddAppointment()** only tested that a valid appointment could be added and retrieved, while **testAddDuplicateAppointmentId()** only tested the rejection of a duplicate ID. This separation meant that when a test failed, it pointed immediately to the specific behavior that broke rather than requiring me to dig through a combined test to figure out what went wrong. Short, focused test methods also made the test suite faster to run since there was no unnecessary setup or teardown happening within each method.

Reflection

Testing Techniques Employed

The primary testing technique I used throughout this project was black-box testing, specifically equivalence partitioning and boundary value analysis. Black-box testing means the tests are written based on what the code is supposed to do according to its requirements, without looking at or relying on the internal implementation. For each service, I identified valid input categories and invalid input categories, and wrote tests to cover both.

Equivalence partitioning meant grouping inputs into classes that should all produce the same result. For example, any contact ID longer than ten characters should be rejected, so I did not need to test every possible length above ten. I just needed one test that represented that entire invalid partition. Boundary value analysis meant paying attention to the edges of those partitions. A ten-character ID should be accepted, and an eleven-character ID should not. Testing at those exact boundaries is where bugs tend to hide, and it gave my tests more precision than just testing random values (Guru99, 2024).

I also used regression-style thinking when I ran the full test suite repeatedly after making changes. Each time I added a new test or modified a method, I reran all tests to confirm I had not broken anything that was already passing. This is a characteristic of regression testing, where you verify that existing functionality holds after changes are introduced.

Other Testing Techniques Not Used

White-box testing is a technique I did not use for this project. White-box testing, also called structural or glass-box testing, requires the tester to have detailed knowledge of the internal logic of the code and to design tests that target specific code paths, branches, and conditions inside the implementation. My tests were written based on the requirements specification, not by reading through the service classes line by line and writing tests for each branch I found there.

Integration testing is another technique I did not apply. Integration testing examines how multiple components interact with each other when they are combined. In this project, each service was tested in isolation. There was no test that called `ContactService` and then passed data to `AppointmentService`, or that connected any of these services to a database, a UI layer, or an external system. The scope of the project did not require it, but in a real deployment those interactions would need to be tested separately.

Performance testing and load testing were also not part of this project. These techniques evaluate how software behaves under high volumes of requests or under resource constraints, and they require a different kind of test tooling than JUnit. For a production mobile application,

you would want to know how the `ContactService` handles thousands of simultaneous additions, but that was outside the scope of what was needed here.

Practical Uses and Implications of Each Technique

Black-box testing with equivalence partitioning and boundary value analysis is most useful during development when requirements are well-defined. It keeps testers focused on what the system is supposed to do rather than how it does it, which means tests remain valid even if the internal implementation changes. The implication is that this approach works best in projects where requirements are documented clearly upfront, which is common in structured environments but harder to apply in rapidly changing agile sprints where requirements shift frequently.

White-box testing is most valuable when code complexity is high and there is a risk that certain internal paths will never be triggered by requirement-based tests alone. It is commonly used during code reviews and security audits because it can surface logic errors and unreachable code that black-box testing would miss entirely. The trade-off is that white-box tests are tightly coupled to the implementation, so any refactor can break them even if the external behavior is unchanged.

Integration testing is critical in any system where components hand data off to each other, which describes most real-world software. Without integration tests, a system can pass all of its unit tests and still fail in production because two components that each work correctly on their own make incompatible assumptions about how data is formatted or sequenced. For a mobile application like the one in this project, integration tests would be essential before deployment to catch those kinds of mismatches.

Performance and load testing are most relevant in consumer-facing systems where unpredictable traffic is expected. The practical implication is that skipping these in development often leads to failures at scale that are very expensive to fix after the fact, which is why they are usually built into the later stages of a release pipeline.

Mindset: Caution

Caution was something I had to actively apply throughout this project, particularly when thinking about how the services interacted with their data structures and how my tests could unintentionally pass for the wrong reasons. One area where I exercised caution was in the appointment date validation. The requirement was that appointment dates cannot be in the past, which sounds straightforward, but it introduced a time-dependency into the tests. If I had hardcoded a date like January 1, 2025, that test would have passed initially and then started failing automatically as time moved forward. I recognized that risk and addressed it by writing the **futureDate()** helper method, which calculates a date dynamically based on the current system time. That small decision reflects the kind of caution that comes from thinking through the interrelationships in the code rather than just writing the most obvious implementation.

I also exercised caution around test isolation. It would have been easy to write tests that shared state, where one test added a record and a later test relied on that record still being there. That approach creates hidden dependencies between tests, and when one test fails it can cause a cascade of failures in tests that appear unrelated. Using **@BeforeEach** to reinitialize the service before every test was my way of guarding against that. It forced each test to be fully self-contained and prevented me from making assumptions about what prior tests had done.

Mindset: Bias

Limiting bias in testing is genuinely difficult, especially in a project where I was both the developer and the tester. When you write the code yourself, you naturally have a mental model of how it works, and that model shapes which inputs you think to test. The bias risk is that you end up testing the scenarios you already know will work and unconsciously avoid the ones where your assumptions might be wrong.

I tried to counteract this by testing failure cases as deliberately as I tested success cases. For every operation that should succeed, I also wrote a test for a condition that should fail. That symmetry was one way of pushing myself past the confirmation bias that comes with testing your own code. For example, in `ContactServiceTest` I wrote **`testAddNullContact()`**, which verifies that **`service.addContact(null)`** throws an exception. That test catches a failure mode I might have overlooked if I had only tested inputs I expected to be valid.

If I were responsible for testing code written by someone else, bias would still be a concern but in a different way. I might bring assumptions from my own coding habits that do not match the original developer's intent, or I might focus on areas I find more interesting and spend less time on simpler-looking code that is actually riskier. The honest answer is that no single tester eliminates bias entirely, which is why peer reviews and independent QA teams exist in professional settings.

Mindset: Discipline

Discipline in software engineering means maintaining consistent standards even when it is tempting to cut corners, and it is directly connected to avoiding technical debt. Technical debt is what accumulates when shortcuts taken in development have to be paid for later through

rework, bug fixes, or system failures. In this project, discipline showed up in a few specific ways.

One was keeping test methods short and single-purpose. It would have been faster to write one large test method that added a contact, updated it, deleted it, and checked all the edge cases in sequence. But that kind of test is harder to maintain, harder to debug when it fails, and harder to extend when requirements change. Keeping each test focused on one behavior required more methods overall, but it makes the test suite significantly easier to work with over time.

Another was not skipping tests for operations that seemed obvious. It might seem unnecessary to write a test confirming that `assertNotNull(service.getAppointment("A1"))` passes after adding an appointment, but that test exists to document expected behavior and to catch regressions if the add logic is ever changed. Skipping it to save time is exactly the kind of shortcut that creates technical debt, because the assumption that it works replaces the guarantee that it has been verified.

As a software engineering professional, I plan to avoid technical debt by treating tests as a required deliverable rather than an optional add-on. Writing tests after the fact is harder and less reliable than writing them alongside the code, and deferring them almost always means they never get written at all. Building the habit of testing consistently from the start is the discipline that prevents the kind of accumulated shortcuts that eventually become serious problems in production systems.

References

Guru99. (2024). *Boundary value analysis and equivalence partitioning*. <https://www.guru99.com/equivalence-partitioning-boundary-value-analysis.html>

JUnit Team. (2024). *JUnit 5 user guide*. <https://junit.org/junit5/docs/current/user-guide/>